

Gestor Devs

Gerenciador de Tarefas Simples

Documento Técnico

Disciplina de Programação III — Unoesc Chapecó
Arthur G. V. da Rocha e Maria Eduarda Rodrigues
2026

1. Nome do Projeto

Gestor Dev's — Gerenciador de Tarefas Simples.

Aplicação web composta por uma API REST (Node.js + Express + PostgreSQL) e um frontend em HTML, CSS e JavaScript puro, que reproduz, de forma simplificada, o fluxo de um quadro de tarefas estilo Jira: cadastro de usuários, criação de tarefas e organização visual por status em um board kanban.

2. Objetivo da Solução

O projeto tem como objetivo aplicar, na prática, conceitos de arquitetura de software e padrões de projeto (design patterns) na construção de uma API REST, integrada a um banco de dados relacional e a uma interface visual funcional.

De forma específica, a solução permite:

- Cadastro e autenticação simples de usuários;
- Criação, edição, exclusão e listagem de tarefas;
- Organização das tarefas em três status (To Do, In Progress, Done);
- Atribuição de uma tarefa a um usuário responsável;
- Visualização das tarefas em um quadro kanban, com movimentação entre colunas via drag and drop.

3. Tecnologias Utilizadas

Categoria	Tecnologia
Backend	Node.js + Express 5
Banco de dados	PostgreSQL (via driver pg)
Autenticação	bcryptjs (hash de senha)
Configuração	dotenv (variáveis de ambiente)
CORS	cors (liberação de requisições do frontend)
Frontend	HTML5 + CSS3 + JavaScript (ES6+), sem frameworks
Comunicação	Fetch API (REST/JSON)
Ferramentas	nodemon (ambiente de desenvolvimento)

4. Arquitetura Adotada

O backend segue uma arquitetura em camadas (Layered Architecture), separando claramente responsabilidades: a camada HTTP (rotas e controllers), a camada de regras de negócio (services) e a camada de acesso a dados (repositories). Essa separação reduz o acoplamento entre as partes do sistema e facilita testes, manutenção e evolução do código.

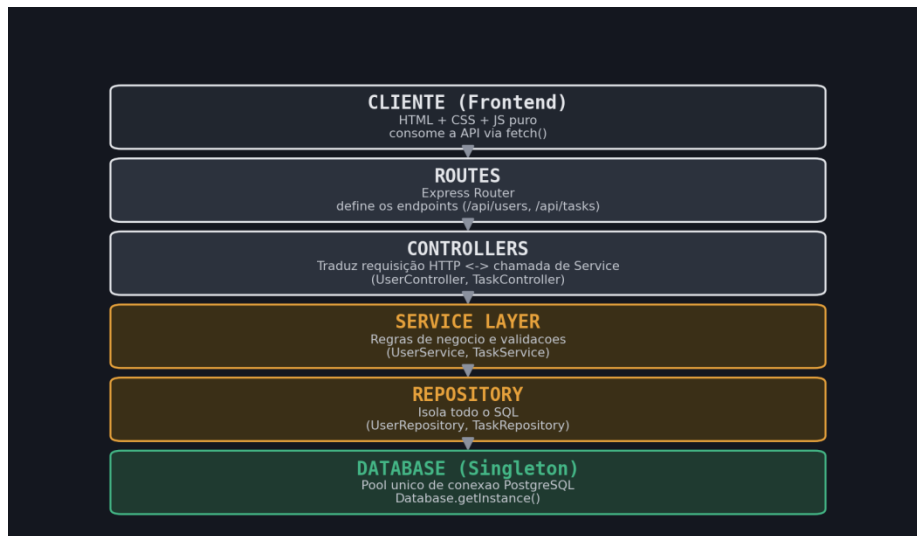


Figura 1 — Fluxo de camadas da aplicação

Fluxo de uma requisição:

- 1. O cliente (frontend) faz uma requisição HTTP para a API;
- 2. As Routes direcionam a requisição ao Controller correspondente;
- 3. O Controller delega a operação ao Service;
- 4. O Service aplica as regras de negócio e chama o Repository;
- 5. O Repository executa o SQL no banco, através da conexão Singleton;
- 6. O resultado percorre o caminho inverso até a resposta HTTP.

5. Explicação dos Design Patterns

5.1 Singleton

Garante que toda a aplicação compartilhe uma única instância da conexão com o banco de dados, evitando a criação de múltiplos pools de conexão desnecessários.

Implementado em src/config/database.js, na classe Database. O método getInstance() verifica se já existe uma instância criada; em caso positivo, a reutiliza; caso contrário, cria a primeira e a guarda em um campo estático privado.

5.2 Repository

Isola todo o acesso a dados (SQL) em classes dedicadas, separando a forma como os dados são persistidos das regras de negócio que os utilizam. Caso o banco de dados seja substituído no futuro, apenas essa camada precisaria ser alterada.

Implementado em `UserRepository` e `TaskRepository` (`src/repositories/`), com métodos como `findAll`, `findById`, `create`, `update` e `delete`.

5.3 Service Layer

Concentra as regras de negócio da aplicação — validações, conferência de duplicidade, hashing de senha, verificação de status válido — mantendo os `Controllers` simples e os `Repositories` livres de lógica de negócio.

Implementado em `UserService` e `TaskService` (`src/services/`). Exemplo de regra aplicada: o status de uma tarefa só pode ser `todo`, `in_progress` ou `done`; qualquer outro valor é rejeitado antes de chegar ao banco.

6. Estrutura do Banco de Dados

O banco é composto por duas tabelas relacionadas: `users` e `tasks`, em um relacionamento de 1 para N (um usuário pode ser responsável por várias tarefas).

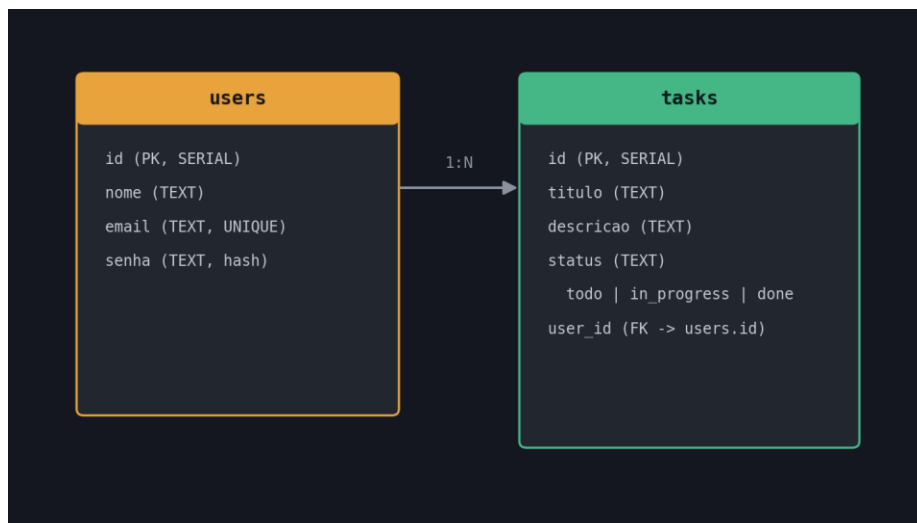


Figura 2 — Diagrama ER simplificado

Tabela	Coluna	Tipo	Observação
users	id	SERIAL	Chave primária
	nome	TEXT	Obrigatório
	email	TEXT	Único, obrigatório
	senha	TEXT	Armazenada com hash (bcrypt)
tasks	id	SERIAL	Chave primária
	titulo	TEXT	Obrigatório
	descricao	TEXT	Opcional
	status	TEXT	todo in_progress done

	user_id	INTEGER	Chave estrangeira -> users.id
--	---------	---------	-------------------------------

7. Prints da Aplicação

As telas abaixo foram capturadas a partir da interface real do projeto.

7.1 Tela de Login

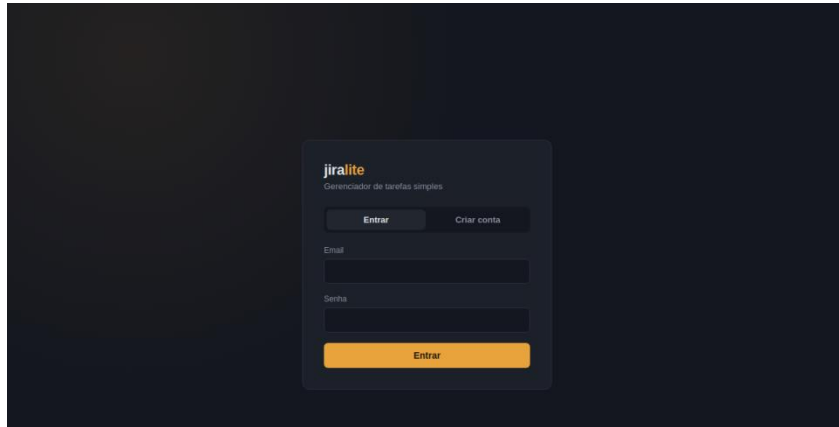


Figura 3 — Tela de login

7.2 Tela de Cadastro

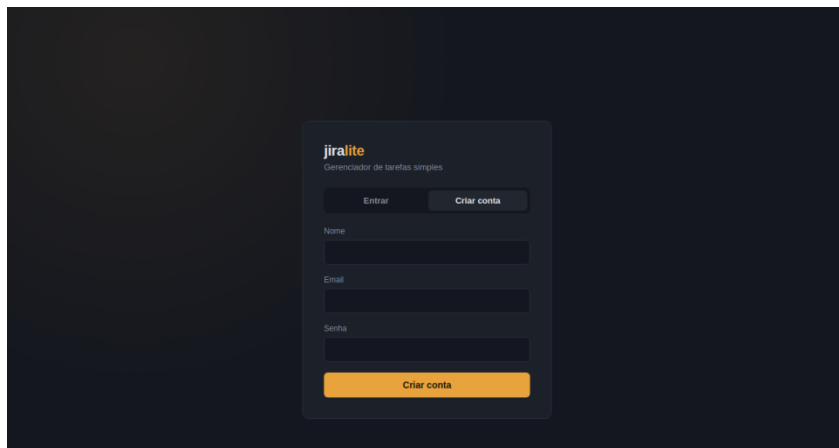


Figura 4 — Tela de criação de conta

7.3 Board Kanban

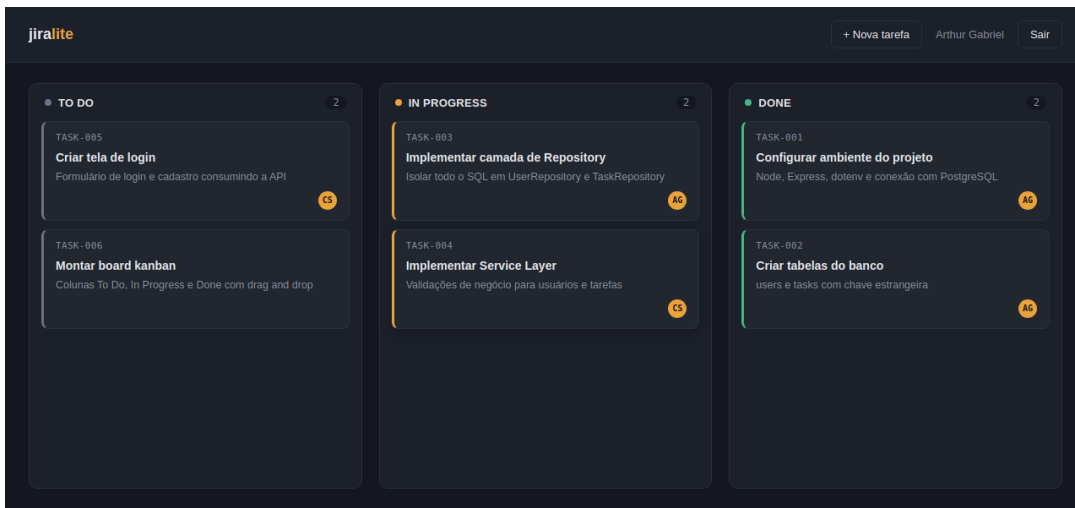


Figura 5 — Board com tarefas distribuídas entre To Do, In Progress e Done

7.4 Criação de Tarefa

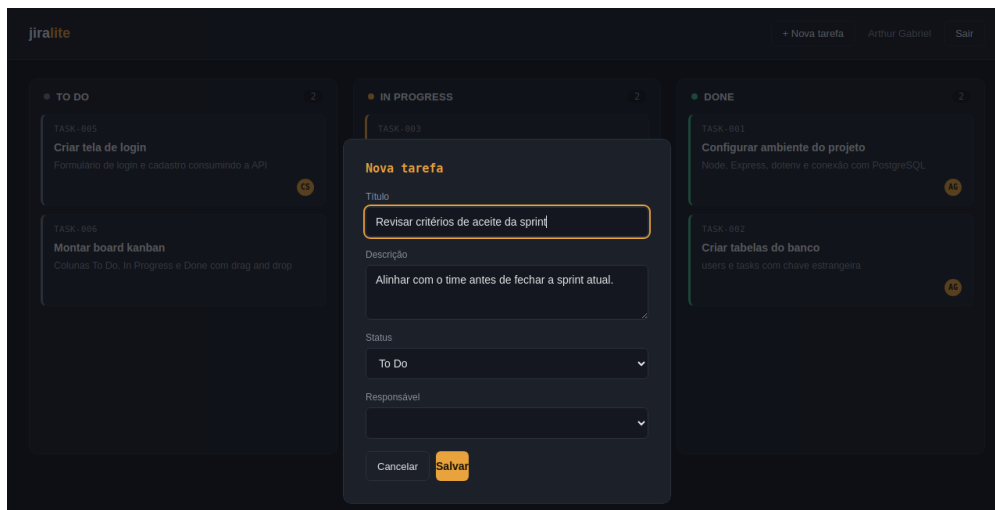


Figura 6 — Modal de criação/edição de tarefa

8. Explicação da API

Todas as rotas têm como prefixo /api e respondem em formato JSON.

8.1 Usuários

Método	Rota	Descrição
POST	/api/users/register	Cria um novo usuário (senha é armazenada com hash)
POST	/api/users/login	Autentica um usuário pelo email e senha
GET	/api/users	Lista todos os usuários

GET	/api/users/:id	Busca um usuário pelo id
PUT	/api/users/:id	Atualiza nome/email de um usuário
DELETE	/api/users/:id	Remove um usuário

8.2 Tarefas

Método	Rota	Descrição
POST	/api/tasks	Cria uma nova tarefa
GET	/api/tasks	Lista todas as tarefas (aceita ?status=)
GET	/api/tasks/:id	Busca uma tarefa pelo id
PUT	/api/tasks/:id	Atualiza todos os campos de uma tarefa
PATCH	/api/tasks/:id/status	Atualiza apenas o status (usado no drag and drop)
DELETE	/api/tasks/:id	Remove uma tarefa

Exemplo de requisição — criar tarefa:

```
POST /api/tasks {  "titulo": "Criar tela de login",  "descricao": "Formulario consumindo a API",  "status": "todo",  "user_id": 2 }
```

9. Dificuldades Encontradas

Durante o desenvolvimento, alguns problemas precisaram ser identificados e corrigidos:

-